

Name: Dwijesh Mistry
Class / Batch: TY4 / B
Roll no: 38

EXPERIMENT NO – 5

Aim: To implement the Apriori algorithm for Association Rule Mining and to identify frequent itemsets and strong association rules from a transactional dataset.

Introduction: Association Rule Mining is a data mining technique used to discover relationships and patterns among variables in large datasets. It is widely applied in areas such as market basket analysis, recommendation systems, and customer behavior analysis. The Apriori algorithm is one of the most commonly used methods for mining frequent itemsets and generating association rules. It operates on the principle known as the *Apriori property*, which states:

“If an itemset is frequent, then all of its subsets must also be frequent.”

This property helps reduce the computational complexity by eliminating candidate itemsets that are unlikely to be frequent. The algorithm follows a level-wise approach, generating candidate itemsets and pruning those that do not satisfy a minimum support threshold.

Procedure:

1. Prepare the dataset consisting of multiple transactions, where each transaction contains a set of items.
2. Define the minimum support and minimum confidence thresholds.
3. Generate the list of all unique items present in the dataset.
4. Calculate the support for each individual item.
5. Eliminate items that do not meet the minimum support threshold.
6. Generate candidate itemsets by combining frequent items.
7. Calculate the support for these candidate itemsets.
8. Prune itemsets that do not satisfy the minimum support.
9. Repeat the process of candidate generation and pruning until no further itemsets can be formed.
10. Generate association rules from the frequent itemsets.
11. Calculate the confidence for each rule.
12. Select the rules that satisfy the minimum confidence threshold.

Program Codes:

```
from itertools import combinations
```

```
# Step 1: Sample dataset (each transaction is a list of items)
transactions = [
    ['milk', 'bread', 'butter'],
    ['bread', 'butter'],
```

```
['milk', 'bread'],  
['milk', 'butter'],  
['bread', 'butter']  
]
```

Minimum support threshold (e.g., 0.4 = appears in 40% of transactions)

min_support = 0.4

min_confidence = 0.6

Step 2: Get all unique items

def get_unique_items(transactions):

items = set()

for transaction in transactions:

for item in transaction:

items.add(item)

return list(items)

Step 3: Calculate support of an itemset

def calculate_support(itemset, transactions):

count = 0

for transaction in transactions:

if set(itemset).issubset(set(transaction)):

count += 1

return count / len(transactions)

Step 4: Generate frequent itemsets

def apriori(transactions, min_support):

items = get_unique_items(transactions)

frequent_itemsets = []

Start with single items (k=1)

k = 1

current_itemsets = [[item] for item in items]

while current_itemsets:

valid_itemsets = []

for itemset in current_itemsets:

support = calculate_support(itemset, transactions)

if support >= min_support:

valid_itemsets.append(itemset)

frequent_itemsets.append((itemset, support))

```

# Generate next level itemsets (k+1)
next_itemsets = []
for i in range(len(valid_itemsets)):
    for j in range(i + 1, len(valid_itemsets)):
        union = list(set(valid_itemsets[i]) | set(valid_itemsets[j]))
        if len(union) == k + 1 and union not in next_itemsets:
            next_itemsets.append(union)

current_itemsets = next_itemsets
k += 1

return frequent_itemsets

```

Step 5: Generate association rules

```

def generate_rules(frequent_itemsets, transactions, min_confidence):
    rules = []

    for itemset, support in frequent_itemsets:
        if len(itemset) > 1:
            # Try all possible splits
            for i in range(1, len(itemset)):
                for antecedent in combinations(itemset, i):
                    antecedent = list(antecedent)
                    consequent = list(set(itemset) - set(antecedent))

                    support_antecedent = calculate_support(antecedent, transactions)
                    confidence = support / support_antecedent

                    if confidence >= min_confidence:
                        rules.append((antecedent, consequent, confidence))

    return rules

```

#Run the algorithm

```

frequent_itemsets = apriori(transactions, min_support)
rules = generate_rules(frequent_itemsets, transactions, min_confidence)

```

#Display results

```

print("Frequent Itemsets:")
for itemset, support in frequent_itemsets:
    print(f'{itemset} -> Support: {round(support, 2)}')

```

```
print("\nAssociation Rules:")
for antecedent, consequent, confidence in rules:
    print(f"{antecedent} -> {consequent} (Confidence: {round(confidence, 2)})")
```

Implementation/Output snap shot:

```
Frequent Itemsets:
['milk'] -> Support: 0.6
['bread'] -> Support: 0.8
['butter'] -> Support: 0.8
['milk', 'bread'] -> Support: 0.4
['milk', 'butter'] -> Support: 0.4
['bread', 'butter'] -> Support: 0.6

Association Rules:
['milk'] -> ['bread'] (Confidence: 0.67)
['milk'] -> ['butter'] (Confidence: 0.67)
['bread'] -> ['butter'] (Confidence: 0.75)
['butter'] -> ['bread'] (Confidence: 0.75)
```

Conclusion: The Apriori algorithm is an effective method for identifying frequent itemsets and generating association rules from large datasets. By applying support and confidence measures, it filters out insignificant patterns and retains only meaningful relationships. This algorithm is particularly useful in real-world applications such as retail analysis, recommendation systems, and decision support systems, where understanding item relationships is essential.

Review Questions:

1. What is the Apriori algorithm in Association Rule Mining?

ANS: The Apriori algorithm is a fundamental technique in data mining used to identify frequent itemsets and generate association rules from transactional data. It works by iteratively expanding itemsets and pruning those that do not meet a specified minimum support threshold. The algorithm uses the Apriori property to improve efficiency by reducing the number of candidate itemsets.

2. What is the significance of support, confidence, and lift in Apriori?

ANS:

Support:

Support measures how frequently an itemset appears in the dataset. It is calculated as the

ratio of transactions containing the itemset to the total number of transactions. It helps in identifying frequent itemsets.

Formula:

$\text{Support}(A) = (\text{Number of transactions containing } A) / (\text{Total number of transactions})$

Confidence:

Confidence measures the reliability of an association rule. It indicates how often the consequent appears in transactions that contain the antecedent.

Formula:

$\text{Confidence}(A \rightarrow B) = \text{Support}(A \cup B) / \text{Support}(A)$

Lift:

Lift measures the strength of an association rule compared to random chance. It indicates whether the occurrence of one item increases the likelihood of the occurrence of another.

Formula:

$\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) / \text{Support}(B)$

Interpretation:

- Lift = 1: No association between items
- Lift > 1: Positive association
- Lift < 1: Negative association